
File System (2)

File system in xv6

Recall: xv6 file system layer

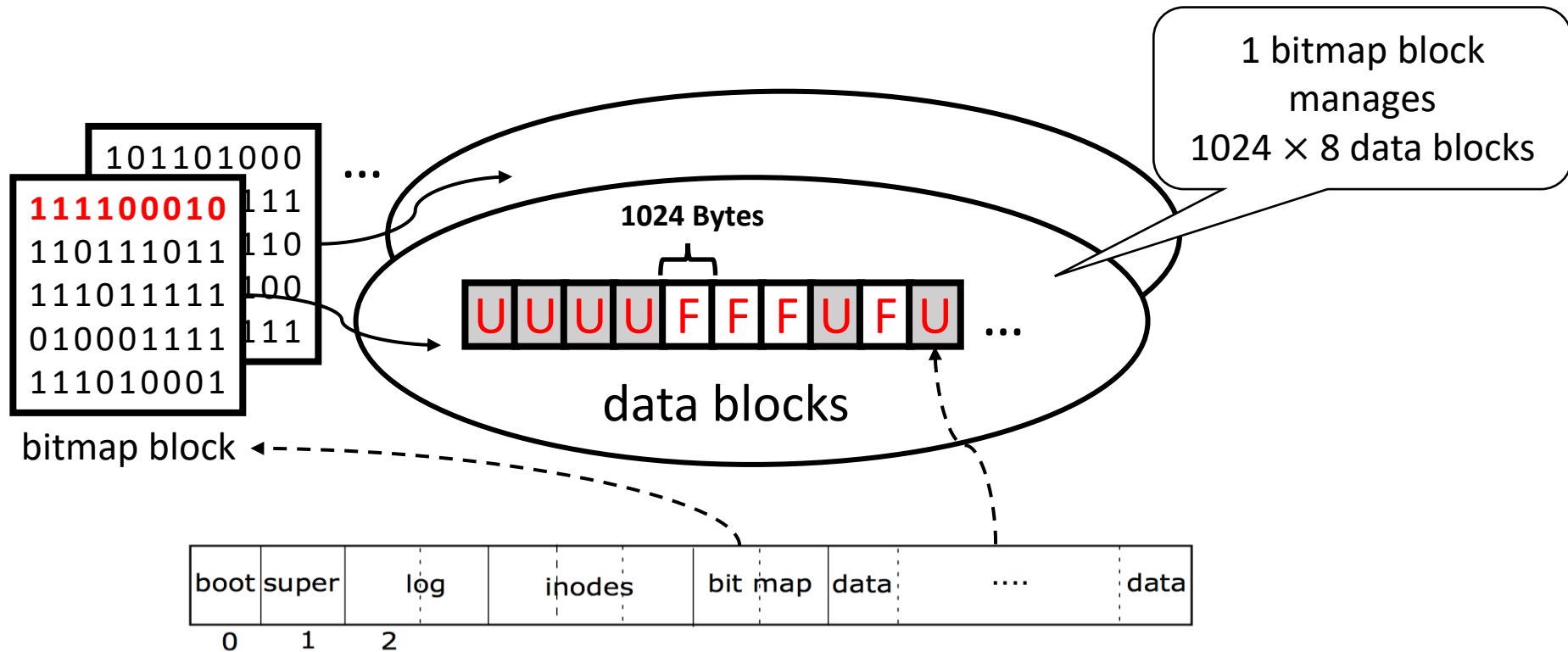
File descriptor	← Abstraction for files, pipes, and devices
Pathname	← Resolves paths
Directory	← Directories as lists of name-number pairs
Inode	← Each file represented by an inode with a unique i-number
Logging	← Ensures atomic updates to multiple blocks
Buffer cache	← Caches and synchronizes access to disk blocks
Disk	← Reads and writes to the virtio disk

Inode Layer

File descriptor	← Abstraction for files, pipes, and devices
Pathname	← Resolves paths
Directory	← Directories as lists of name-number pairs
Inode	← Each file represented by an inode with a unique i-number
Logging	← Ensures atomic updates to multiple blocks
Buffer cache	← Caches and synchronizes access to disk blocks
Disk	← Reads and writes to the virtio disk

Block Allocator

- Blocks are allocated from a free pool.
- Bitmap on disk manages in-use/free data blocks, one bit per block.



balloc

- balloc allocates one block from the pool using bitmap.

```
$ vim kernel/fs.c
```

```
66 static uint
67 balloc(uint dev)
68 {
...
72 bp = 0;
73 for(b = 0; b < sb.size; b+= BPB){
74     bp = bread(dev, BBLOCK(b, sb));
75     for(bi = 0; bi < BPB && b + bi < sb.size; bi++){
76         m = 1 << (bi % 8);
77         if((bp->data[bi/8] & m) == 0){ // Is block free?
78             bp->data[bi/8] |= m; // Mark block in use.
79             log_write(bp);
80             brelse(bp);
81             bzero(dev, b + bi);
82             return b + bi;
83         }
84     }
85     brelse(bp);
86 }
```

BPB means Bitmap bits
per Block (= BSIZE×8)

BBLOCK gets the block number
where target bit is located

Looks for a block whose
bitmap bit is zero.

balloc

- balloc allocates one block from the pool using bitmap.

```
$ vim kernel/fs.c

66 static uint
67 balloc(uint dev)
68 {
...
72     bp = 0;
73     for(b = 0; b < sb.size; b+= BPB){
74         bp = bread(dev, BBLOCK(b, sb));
75         for(bi = 0; bi < BPB && b + bi < sb.size; bi++){
76             m = 1 << (bi % 8);
77             if((bp->data[bi/8] & m) == 0){ // Is block free?
78                 bp->data[bi/8] |= m; // Mark block in use.
79                 log_write(bp);
80                 brelse(bp);
81                 bzero(dev, b + bi);
82                 return b + bi;
83             }
84         }
85         brelse(bp);
86     }
...

```

Only one process
can use any one bitmap block.

bfree

- bfree reclaims the used block to the pool using bitmap.

```
$ vim kernel/fs.c
```

```
92 static void
93 bfree(int dev, uint b)
94 {
95     struct buf *bp;
96     int bi, m;
97
98     bp = bread(dev, BBLOCK(b, sb));
99     bi = b % BPB;
100     m = 1 << (bi % 8);
101     if((bp->data[bi/8] & m) == 0)
102         panic("freeing free block");
103     bp->data[bi/8] &= ~m;
104     log_write(bp);
105     brelse(bp);
106 }
```

Clears the right bit

Inode layer

- An **inode** is a data structure that stores a file's metadata, such as its size and the location of its data blocks.
- There are two types of inodes
 - **On-disk inode**(`struct dinode`):
 - Stored on disk with a fixed size, located in a contiguous inode block region.
 - **In-memory inode**(`struct inode`):
 - A copy of the on-disk inode along with additional kernel-specific information.
 - Actively accessed inodes are managed in an array called `itable`.
- Interfaces:
 - `iget`, `iput` / `ialloc`, `itrunc` / `ilock`, `iunlock` / `iupdate` / `readi`, `writei`

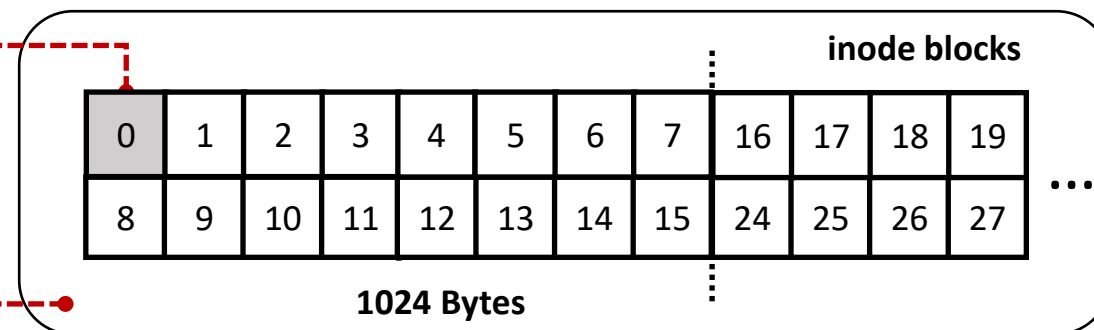
On-disk inode

- Dinode is located in inode section of disk.

```
$ vim kernel/fs.h
```

```
31 // On-disk inode structure
32 struct dinode {
33     short type;           // + 2 bytes
34     short major;          // + 2 bytes
35     short minor;          // + 2 bytes
36     short nlink;          // + 2 bytes
37     uint size;            // + 4 bytes
38     uint addr[NDIRECT+1]; // + 4 * 13 bytes
39 };                       // = 4 * 16 bytes
```

type	free(0)/ file/ directory/ special file(device)
nlink	the number of directory entries that refer to this inode
size	the number of bytes of content in the file
addr	an array of block numbers of the file.



In-memory inode

- Kernel keeps inode only if there are pointers referring to that.

```
$ vim kernel/file.h
```

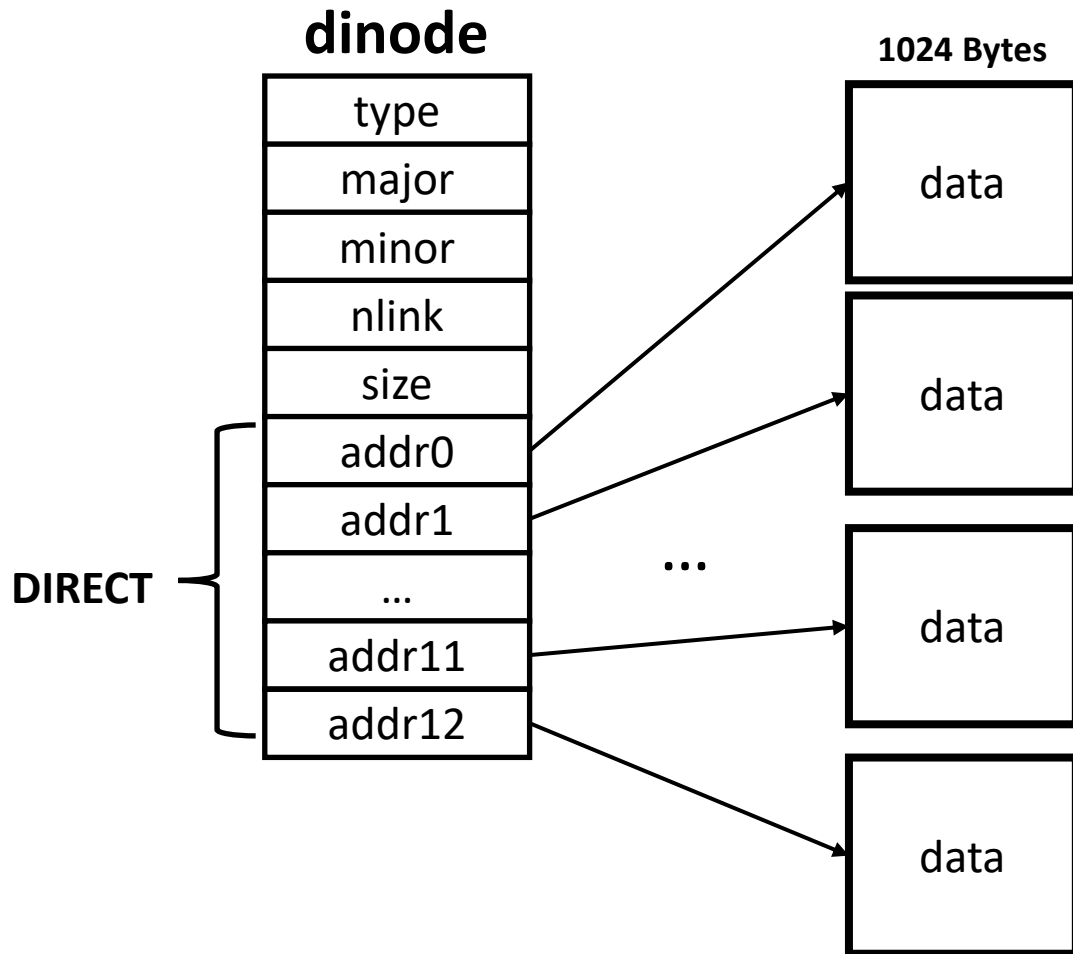
```
16 // in-memory copy of an inode
17 struct inode {
18     uint dev;
19     uint inum;
20     int ref;
21     struct sleeplock lock;
22     int valid;
23
24     short type;
25     short major;
26     short minor;
27     short nlink;
28     uint size;
29     uint addrs[NDIRECT+1];
30 };
```

inum	n-th inode on the disk
ref	the number of <u>pointers</u> referring to the in-memory inode
valid	whether inode is valid (1) or not(0)
	a copy of on-disk inode (dinode)

Pointers could be :

- file descriptors
- current working directory
- transient kernel code
- ...

inode Content



If we use all addrs as **DIRECT**:
Max file size of one inode is
“ $\text{NDIRECT} (13) \times \text{BSIZE}(1024) \text{ Bytes}$ ”

```
$ vim kernel/fs.h
```

```
27 #define NDIRECT 12
```

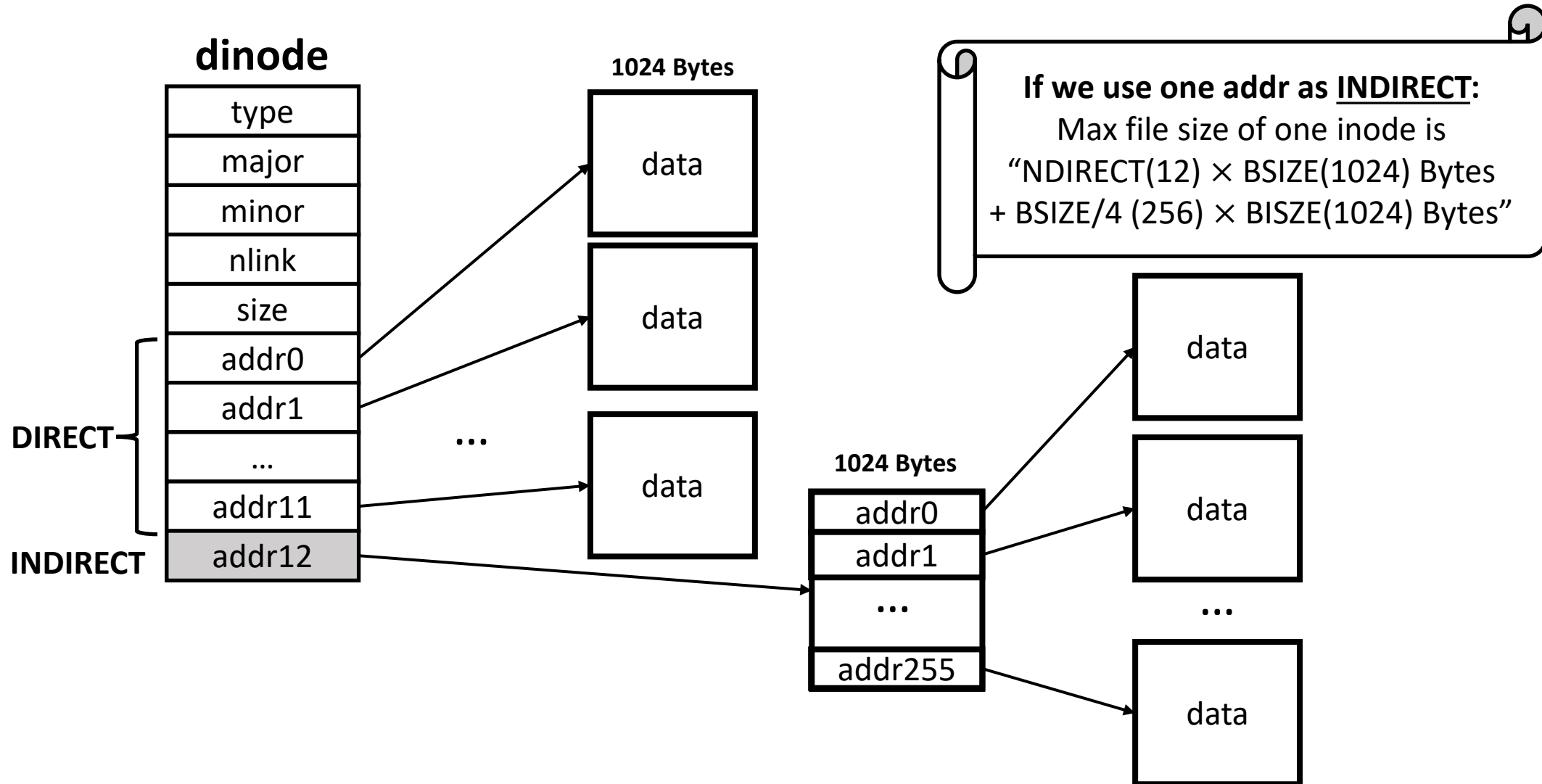
```
$ vim kernel/file.h
```

```
16 // in-memory copy of an inode
17 struct inode {
```

```
...
```

```
28     uint size;
29     uint addrs[NDIRECT+1];
30 };
```

inode Content



bmap

- bmap returns the block number of the *bn*-th data block from the inode *ip*.

DIRECT

```
$ vim kernel/fs.c

405 static uint
406 bmap(struct inode *ip, uint bn)
407 {
408     uint addr, *a;
409     struct buf *bp;
410
411     if(bn < NDIRECT){
412         if((addr = ip->addrs[bn]) == 0){
413             addr = balloc(ip->dev);
414             if(addr == 0)
415                 return 0;
416             ip->addrs[bn] = addr;
417         }
418         return addr;
419     }
420     bn -= NDIRECT;
421     ...
```

INDIRECT

```
$ vim kernel/fs.c

...
422     if(bn < NINDIRECT){
423         // Load indirect block, allocating if necessary.
424         if((addr = ip->addrs[NDIRECT]) == 0){
425             addr = balloc(ip->dev);
426             if(addr == 0)
427                 return 0;
428             ip->addrs[NDIRECT] = addr;
429         }
430         bp = bread(ip->dev, addr);
431         a = (uint*)bp->data;
432         if((addr = a[bn]) == 0){
433             addr = balloc(ip->dev);
434             if(addr){
435                 a[bn] = addr;
436                 log_write(bp);
437             }
438         }
439         brelse(bp);
440         return addr;
441     }
442     ...
```

If *bn* > max file size, then panic.
We can handle this by
second-level indirection.

bmap

- bmap returns the block number of the *bn*-th data block from the inode *ip*.

DIRECT

```
$ vim kernel/fs.c

405 static uint
406 bmap(struct inode *ip, uint bn)
407 {
408     uint addr, *a;
409     struct buf *bp;
410
411     if(bn < NDIRECT){
412         if((addr = ip->addrs[bn]) == 0){
413             addr = balloc(ip->dev);
414             if(addr == 0)
415                 return 0;
416             ip->addrs[bn] = addr;
417         }
418         return addr;
419     }
420     bn -= NDIRECT;
421     ...
```

INDIRECT

```
$ vim kernel/fs.c

...
422 if(bn < NINDIRECT){
423     // Load indirect block, allocating if necessary.
424     if((addr = ip->addrs[NDIRECT]) == 0){
425         addr = balloc(ip->dev);
426         if(addr == 0)
427             return 0;
428         ip->addrs[NDIRECT] = addr;
429     }
430     bp = bread(ip->dev, addr);
431     a = (uint*)bp->data;
432     if((addr = a[bn]) == 0){
433         addr = balloc(ip->dev);
434         if(addr){
435             a[bn] = addr;
436             log_write(bp);
437         }
438     }
439     brelse(bp);
440     return addr;
441 }

...
```

Bmap allocates block on-demand
when the addr entry is zero
(no block is allocated yet).

inode interfaces

ialloc	Allocate a new inode on disk.
itrunc	Deallocate all blocks in an inode on disk.
iget	Get a reference to an inode in memory. (Provides non-exclusive access)
iput	Drop a reference to an inode in memory.
ilock	Lock an inode. (Provides exclusive access)
iunlock	Unlock an inode.
iupdate	Write the modified inode in memory to the inode on disk.
readi	Read data from an inode.
writei	Write data to an inode.

ialloc

- ialloc allocates a new inode. (on-disk)

```
$ vim kernel/fs.c
```

```
199 struct inode*
200 ialloc(uint dev, short type)
201 {
202     int inum;
203     struct buf *bp;
204     struct dinode *dip;
205
206     for(inum = 1; inum < sb.ninodes; inum++){
207         bp = bread(dev, IBLOCK(inum, sb));
208         dip = (struct dinode*)bp->data + inum%IPB;
209         if(dip->type == 0){ // a free inode
210             memset(dip, 0, sizeof(*dip));
211             dip->type = type;
212             log_write(bp); // mark it allocated on the disk
213             brelse(bp);
214             return iget(dev, inum);
215         }
216         brelse(bp);
217     }
218     printf("ialloc: no inodes\n");
219     return 0;
220 }
```

IBLOCK gets the block number
where target inode is located

IPB means inodes per block = BSIZE / sizeof(dinode).

Looks for a free dinode
whose type is set to 0.

ialloc

- ialloc allocates a new inode. (on-disk)

```
$ vim kernel/fs.c
```

```
199 struct inode*
200 ialloc(uint dev, short type)
201 {
202     int inum;
203     struct buf *bp;
204     struct dinode *dip;
205
206     for(inum = 1; inum < sb.ninodes; inum++){
207         bp = bread(dev, IBLOCK(inum, sb));
208         dip = (struct dinode*)bp->data + inum%IPB;
209         if(dip->type == 0){ // a free inode
210             memset(dip, 0, sizeof(*dip));
211             dip->type = type;
212             log_write(bp); // mark it allocated on the disk
213             brelse(bp);
214             return iget(dev, inum);
215         }
216         brelse(bp);
217     }
218     printf("ialloc: no inodes\n");
219     return 0;
220 }
```

Only one process
can use any one block
containing inodes.

Return active inode entry
from the inode cache.

itrunc

- itrunc(ate) deallocates all blocks in inode and reset the size to 0.

\$ vim kernel/fs.c	DIRECT	\$ vim kernel/fs.c	INDIRECT
<pre>448 static void 449 itrunc(struct inode *ip) 450 { 451 int i, j; 452 struct buf *bp; 453 uint *a; 454 455 for(i = 0; i < NDIRECT; i++){ 456 if(ip->addrs[i]){ 457 bfree(ip->dev, ip->addrs[i]); 458 ip->addrs[i] = 0; 459 } 460 } 461 ...</pre>		<pre>... 462 if(ip->addrs[NDIRECT]){ 463 bp = bread(ip->dev, ip->addrs[NDIRECT]); 464 a = (uint*)bp->data; 465 for(j = 0; j < NINDIRECT; j++){ 466 if(a[j]) 467 bfree(ip->dev, a[j]); 468 } 469 brelse(bp); 470 bfree(ip->dev, ip->addrs[NDIRECT]); 471 ip->addrs[NDIRECT] = 0; 472 } 473 474 ip->size = 0; 475 iupdate(ip); 476 }</pre>	

1. Deallocates direct data blocks.
2. Deallocates indirect data blocks.
3. Deallocates the indirect block.

iget

- iget returns the active inode entry from inode cache. (in-memory)

```
$ vim kernel/fs.c
```

```
247 static struct inode*
248 iget(uint dev, uint inum)
249 {
250     struct inode *ip, *empty;
251
252     acquire(&itable.lock);
253
254     // Is the inode already in the table?
255     empty = 0;
256     for(ip = &itable.inode[0]; ip < &itable.inode[NINODE]; ip++){
257         if(ip->ref > 0 && ip->dev == dev && ip->inum == inum){
258             ip->ref++;
259             release(&itable.lock);
260             return ip;
261         }
262         if(empty == 0 && ip->ref == 0)
263             empty = ip;
264     }
```

Inode table(cache)
contains active inodes
whose 'ref' value is > 0.

If cache hits:
Increase the 'ref' value
and return the inode.

Remember empty
slot for cache miss

iget

- iget returns the active inode entry from inode cache. (in-memory)

```
$ vim kernel/fs.c
```

```
...  
266 // Recycle an inode cache entry.  
267 if(empty == 0)  
268     panic("iget: no inodes");  
269  
270 ip = empty;  
271 ip->dev = dev;  
272 ip->inum = inum;  
273 ip->ref = 1;  
274 ip->valid = 0;  
275 release(&itable.lock);  
276  
277 return ip;  
278 }
```

If cache misses:
Recycle an inode cache entry
using remembered empty slot.
Set valid 0 to invalidate the inode.

Note that, iget provides **non-exclusive access**:
There can be many pointers to the same inode.
It is possible that returned inode entry is not valid
yet. You need to ilock to use this inode, which
provides exclusive access to the inode in memory.

iput

- iput decreases the reference count of an active inode entry.

```
$ vim kernel/fs.c
```

```
337 void
338 iput(struct inode *ip)
339 {
340     acquire(&itable.lock);
341
342     if(ip->ref == 1 && ip->valid && ip->nlink == 0){
343         ...
347         acquiresleep(&ip->lock);
348         release(&itable.lock);
349
350         itrunc(ip);
351         ip->type = 0;
352         iupdate(ip);
353         ip->valid = 0;
354
355         releasesleep(&ip->lock);
356         acquire(&itable.lock);
357     }
358
359     ip->ref--;
360     release(&itable.lock);
361 }
362
363 }
```

If it is the last reference:

- (1) Truncate the inode on disk.
- (2) Free the inode on disk.
(Set type to 0)
- (3) Invalidate the inode in memory from inode cache to reuse this empty slot for other accessors
(reference count is set to 0)

ilock

- `ilock` locks the inode. It reads the inode from disk if necessary.

```
$ vim kernel/fs.c
```

```
293 void
294 ilock(struct inode *ip)
295 {
...
302     acquiresleep(&ip->lock);
303
304     if(ip->valid == 0){
305         bp = bread(ip->dev, IBLOCK(ip->inum, sb));
306         dip = (struct dinode*)bp->data + ip->inum%IPB;
307         ip->type = dip->type;
308         ip->major = dip->major;
309         ip->minor = dip->minor;
310         ip->nlink = dip->nlink;
311         ip->size = dip->size;
312         memmove(ip->addrs, dip->addrs, sizeof(ip->addrs));
313         brelse(bp);
314         ip->valid = 1;
315         if(ip->type == 0)
316             panic("ilock: no type");
317     }
318 }
```

Only one process can proceed,
while others sleep on that inode.

Read the inode from disk
if the inode is not valid yet.

Validate the inode entry.

iunlock

- `iunlock` unlocks the inode.

```
$ vim kernel/fs.c  
  
321 void  
322 iunlock(struct inode *ip)  
323 {  
324     if(ip == 0 || !holdingsleep(&ip->lock) || ip->ref < 1)  
325         panic("iunlock");  
326  
327     releasesleep(&ip->lock);  
328 }
```

Just wakeup sleeping
processes on that inode.

iupdate

- iupdate updates the inode on disk with the modified inode in memory and writes to disk.

```
$ vim kernel/fs.c

226 void
227 iupdate(struct inode *ip)
228 {
229     struct buf *bp;
230     struct dinode *dip;
231
232     bp = bread(ip->dev, IBLOCK(ip->inum, sb));
233     dip = (struct dinode*)bp->data + ip->inum%IPB;
234     dip->type = ip->type;
235     dip->major = ip->major;
236     dip->minor = ip->minor;
237     dip->nlink = ip->nlink;
238     dip->size = ip->size;
239     memmove(dip->addrs, ip->addrs, sizeof(ip->addrs));
240     log_write(bp);
241     brelse(bp);
242 }
```

Update the dinode contents
with the given inode in memory.

readi

```
$ vim kernel/fs.c
```

```
494 int
495 readi(struct inode *ip, int user_dst, uint64 dst, uint off, uint n)
496 {
497     uint tot, m;
498     struct buf *bp;
499
500     if(off > ip->size || off + n < off)
501         return 0;
502     if(off + n > ip->size)
503         n = ip->size - off;
504
505     for(tot=0; tot<n; tot+=m, off+=m, dst+=m){
506         uint addr = bmap(ip, off/BSIZE);
507         if(addr == 0)
508             break;
509         bp = bread(ip->dev, addr);
510         m = min(n - tot, BSIZE - off%BSIZE);
511         if(either_copyout(user_dst, dst, bp->data + (off % BSIZE), m) == -1) {
512             brelse(bp);
513             tot = -1;
514             break;
515         }
516         brelse(bp);
517     }
518     return tot;
519 }
```

- `readi` reads n bytes data from inode ip , starting from off .

Fail if current offset > file size.
It can read at most the file size
and can't read beyond the file size.

Get data block number of off in the file.

Load the block into buffer.

Copy data from the buffer to dst .

writei

```
$ vim kernel/fs.c
```

```
528 int
529 writei(struct inode *ip, int user_src, uint64 src, uint off, uint n)
530 {
531     uint tot, m;
532     struct buf *bp;
533
534     if(off > ip->size || off + n < off)
535         return -1;
536     if(off + n > MAXFILE*BSIZE)
537         return -1;
538
539     for(tot=0; tot<n; tot+=m, off+=m, src+=m){
540         uint addr = bmap(ip, off/BSIZE);
541         if(addr == 0)
542             break;
543         bp = bread(ip->dev, addr);
544         m = min(n - tot, BSIZE - off%BSIZE);
545         if(either_copyin(bp->data + (off % BSIZE), user_src, src, m) == -1) {
546             brelse(bp);
547             break;
548         }
549         log_write(bp);
550         brelse(bp);
551     }
```

- `writei` writes n bytes data to inode ip , starting from off .

Fail if current offset > file size.
It can write beyond the file size
but can't exceed the max file size.

Get data block number of off in the file.

Load the block into buffer.

Copy data from src to the buffer.

writei

```
$ vim kernel/fs.c
```

```
528 int
529 writei(struct inode *ip, int user_src, uint64 src, uint off, uint n)
530 {
531     uint tot, m;
532     struct buf *bp;
533
534     if(off > ip->size || off + n < off)
535         return -1;
536     if(off + n > MAXFILE*BSIZE)
537         return -1;
538
539     for(tot=0; tot<n; tot+=m, off+=m, src+=m){
540         uint addr = bmap(ip, off/BSIZE);
541         if(addr == 0)
542             break;
543         bp = bread(ip->dev, addr);
544         m = min(n - tot, BSIZE - off%BSIZE);
545         if(either_copyin(bp->data + (off % BSIZE), user_src, src, m) == -1) {
546             brelse(bp);
547             break;
548         }
549         log_write(bp);
550         brelse(bp);
551     }
552     ...
```

- `writei` writes n bytes data to inode ip , starting from off .

```
$ vim kernel/fs.c
```

```
...
553     if(off > ip->size){
554         ip->size = off;
555     }
556     ...
559     iupdate(ip);
560
561     return tot;
562 }
```

Must update the size of inode if the write has extended the file.

Directory layer

File descriptor	← Abstraction for files, pipes, and devices
Pathname	← Resolves paths
Directory	← Directories as lists of name-number pairs
Inode	← Each file represented by an inode with a unique i-number
Logging	← Ensures atomic updates to multiple blocks
Buffer cache	← Caches and synchronizes access to disk blocks
Disk	← Reads and writes to the virtio disk

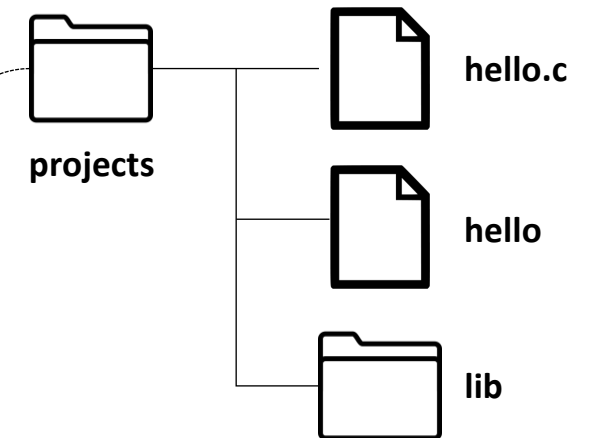
Directory layer

- Directory is a special file (type of T_DIR) containing a sequence of directory entries.
- Directory entry contains a name and an inode number of the file.
- The name of a directory entry cannot exceed DIRSIZE (14).

```
$ vim kernel/fs.h
```

```
58 struct dirent {  
59     ushort inum;  
60     char name[DIRSIZ] __attribute__((nonstring));  
61 };
```

inum	name
5	.
6	..
9	hello.c
11	hello
15	lib



Directory entry lookup

```
$ vim kernel/fs.c
```

```
574 struct inode*
575 dirlookup(struct inode *dp, char *name, uint *poff)
576 {
577     uint off, inum;
578     struct dirent de;
579
580     if(dp->type != T_DIR)
581         panic("dirlookup not DIR");
582
583     for(off = 0; off < dp->size; off += sizeof(de)){
584         if(readi(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
585             panic("dirlookup read");
586         if(de.inum == 0)
587             continue;
588         if(namecmp(name, de.name) == 0){
589             // entry matches path element
590             if(poff)
591                 *poff = off;
592             inum = de.inum;
593             return iget(dp->dev, inum);
594         }
595     }
596
597     return 0;
598 }
```

- **dirlookup()** searches for a directory entry with a given name inside a directory.
- If found :
 - Returns the corresponding inode using **iget()**.
 - Stores the byte offset of the found entry in ***poff**.
- The returned inode is **not locked** because :
 - The caller already holds a lock on the directory.
 - Locking again could cause **deadlock**.

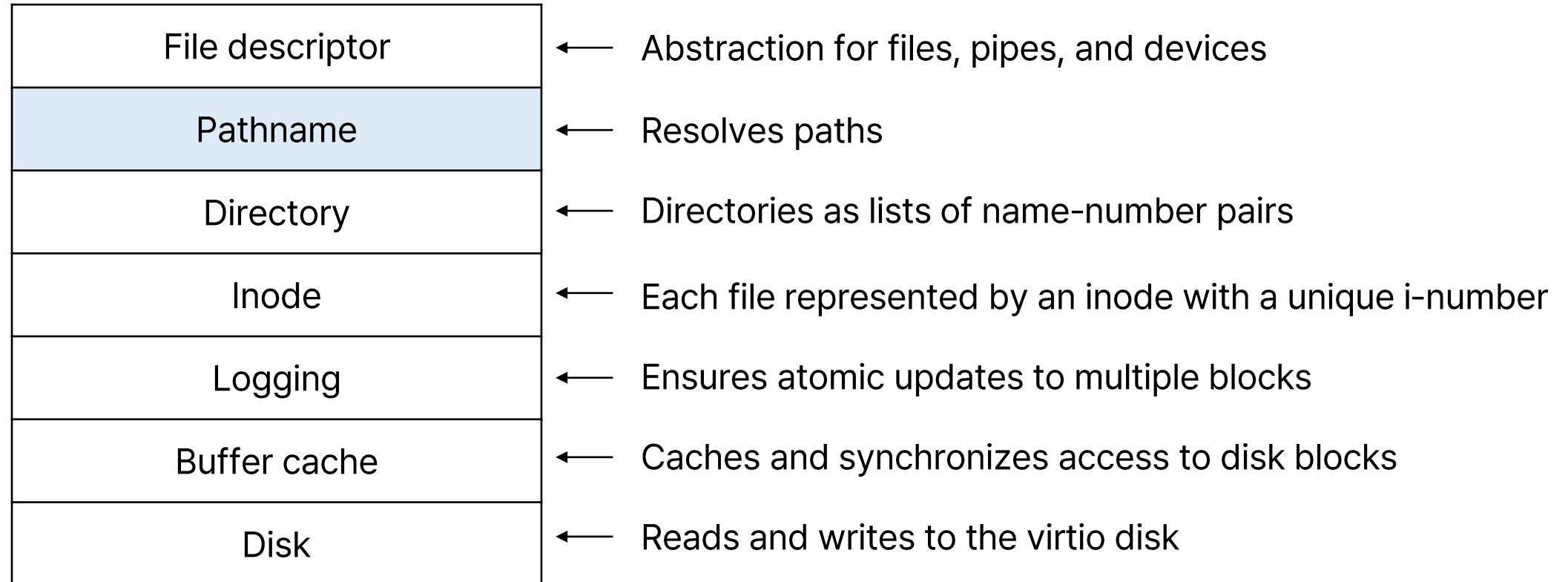
Directory entry insertion

```
$ vim kernel/fs.c
```

```
602 int
603 dirlink(struct inode *dp, char *name, uint inum)
604 {
605     int off;
606     struct dirent de;
607     struct inode *ip;
608
609     // Check that name is not present.
610     if((ip = dirlookup(dp, name, 0)) != 0){
611         iput(ip);
612         return -1;
613     }
614
615     // Look for an empty dirent.
616     for(off = 0; off < dp->size; off += sizeof(de)){
617         if(readi(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
618             panic("dirlink read");
619         if(de.inum == 0)
620             break;
621     }
622
623     strncpy(de.name, name, DIRSIZ);
624     de.inum = inum;
625     if(writei(dp, 0, (uint64)&de, off, sizeof(de)) != sizeof(de))
626         return -1;
627
628     return 0;
629 }
```

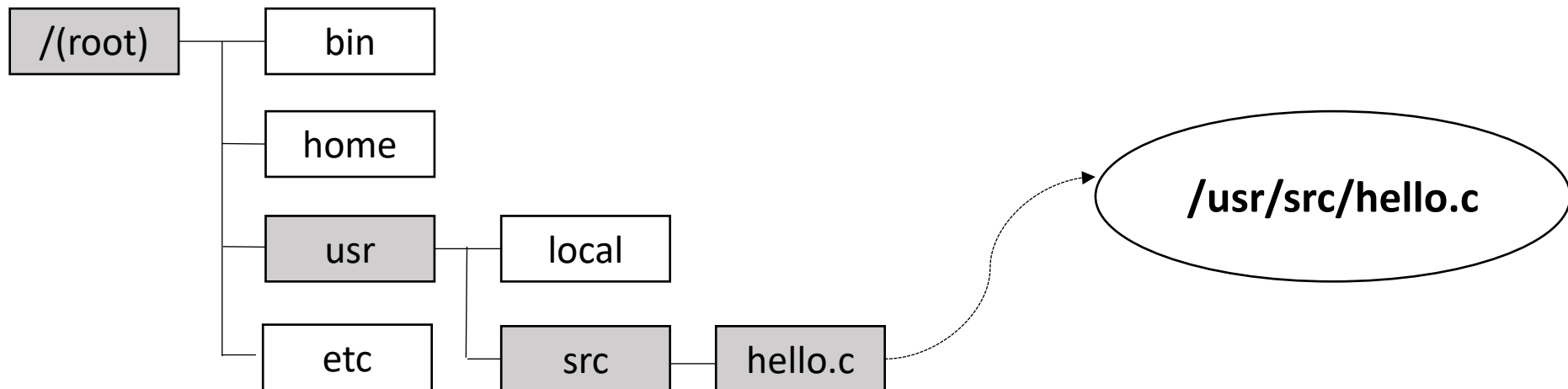
- **dirlink()** adds new entry to a directory.
 - If the name already exists in the directory, it returns an error.
- If the name not exists :
 - **Searches for a free entry(inum == 0) to reuse,**
 - Or appends the new entry at the end of the directory.
- It uses **writei()** to write a struct dirent at the determined offset.

Pathname Layer



Pathname

- Pathname specifies the location of a file.
- Pathname depicts the directory tree hierarchy expressed in a string of characters, with the delimiter character `"/"`.
 - Absolute pathname: Start with `"/"`, providing the full location of the pathname.
 - Relative pathname: Start from some given working directory.



Path name resolution

- **namei(path)** returns the inode of the final component of the given path.
 - ex) for "a/b/c", it returns the inode of "c".
- **nameiparent(path, name)** returns the inode of the parent directory of the final path component.
 - And copies the final component into name.
 - ex) For "a/b/c", it returns the inode of "b" and sets name = "c".
- **namex(path, nameiparent, name)** resolves the pathname by traversing directories step by step.

namex

```
$ vi kernel/fs.c
```

```
674 static struct inode*
675 namex(char *path, int nameparent, char *name)
676 {
677     struct inode *ip, *next;
678
679     if(*path == '/')
680         ip = iget(ROOTDEV, ROOTINO);
681     else
682         ip = idup(myproc()->cwd);
683
684     while((path = skipelem(path, name)) != 0){
685         ilock(ip);
686         if(ip->type != T_DIR){
687             iunlockput(ip);
688             return 0;
689         }
690     }
```

If path starts with "/",
then starts from root. (Absolute path)
Otherwise, starts from current
working directory. (Relative path)

Examples of skipelem:

skipelem("a/bb/c", name) = "bb/c", setting name = "a"
skipelem("//a//bb", name) = "bb", setting name = "a"
skipelem("a", name) = "", setting name = "a"
skipelem("", name) = 0

namex

```
$ vi kernel/fs.c
```

```
684 while((path = skipelem(path, name)) != 0){
```

```
690     if(nameiparent && *path == '\\0'){
```

```
691         // Stop one level early.
```

```
692         iunlock(ip);
```

```
693         return ip;
```

```
694     }
```

```
695     if((next = dirlookup(ip, name, 0)) == 0){
```

```
696         iunlockput(ip);
```

```
697         return 0;
```

```
698     }
```

```
699     iunlockput(ip);
```

```
700     ip = next;
```

```
701 }
```

```
702 if(nameiparent){
```

```
703     iput(ip);
```

```
704     return 0;
```

```
705 }
```

```
706 return ip;
```

```
707 }
```

If this is the last path element,
Nameiparent returns the parent inode
with the name of the last element.

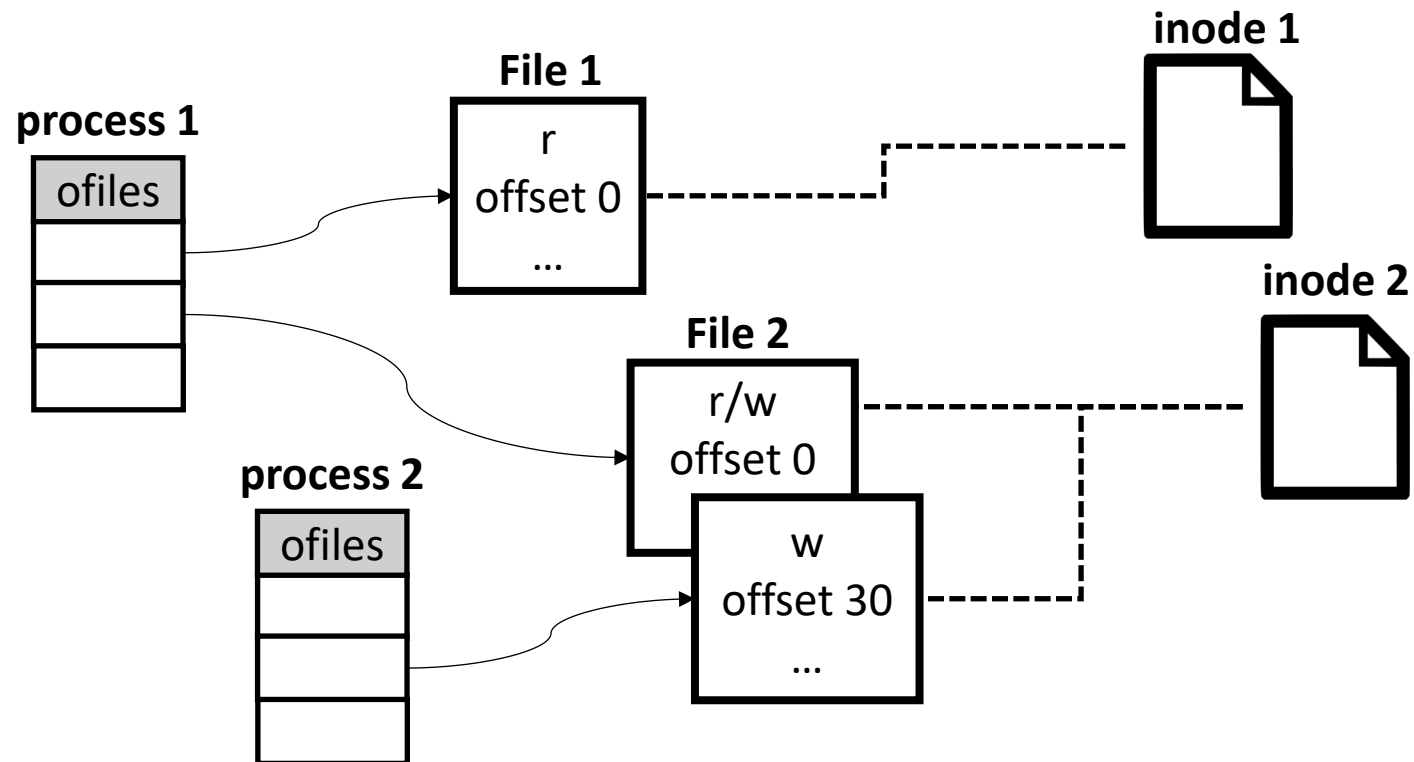
Search the path element using dirlookup
and prepares for the next iteration by
setting ip = next.

File Descriptor Layer

File descriptor	← Abstraction for files, pipes, and devices
Pathname	← Resolves paths
Directory	← Directories as lists of name-number pairs
Inode	← Each file represented by an inode with a unique i-number
Logging	← Ensures atomic updates to multiple blocks
Buffer cache	← Caches and synchronizes access to disk blocks
Disk	← Reads and writes to the virtio disk

File Descriptor

- Most of resources in Unix (console, pipes, sockets, real files, ...) are represented as a file.
- Each process has its own table of open files, called file descriptor table.



File Structure

- File structure is a wrapper of an inode (or a pipe), with an independent i/o offset.
- If multiple processes open the same file individually, each process has its file instance in its file table and has different i/o offset.
- A single open file can appear multiple times in one process's file table (dup) and also in other processes' file table (fork)

```
$ vim kernel/file.h

1 struct file {
2     enum { FD_NONE, FD_PIPE, FD_INODE, FD_DEVICE } type;
3     int ref; // reference count
4     char readable;
5     char writable;
6     struct pipe *pipe;
7     struct inode *ip;
8     uint off;
9     short major;
10 };
```

ftable and Interfaces

```
$ vim kernel/file.h
```

```
1 struct file {
2     enum { FD_NONE, FD_PIPE, FD_INODE, FD_DEVICE } type;
3     int ref; // reference count
4     char readable;
5     char writable;
6     struct pipe *pipe;
7     struct inode *ip;
8     uint off;
9     short major;
10 };
```

```
$ vim kernel/file.c
```

```
17 struct {
18     struct spinlock lock;
19     struct file file[NFILE];
20 } ftable;
```

```
$ vim kernel/proc.h
```

```
85 struct proc {
...
104 struct file *ofile[Nofile];
...
```

- All the open files in the system are managed in a global file table, called `ftable`
- Each process maintains its own array of pointers to these open files.
- Interface
 - `filealloc`
 - `filedup`
 - `fileclose`
 - `fileread`
 - `filewrite`

filealloc

```
$ vim kernel/file.c

29 struct file*
30 filealloc(void)
31 {
32     struct file *f;
33
34     acquire(&ftable.lock);
35     for(f = ftable.file; f < ftable.file + NFILE; f++){
36         if(f->ref == 0){
37             f->ref = 1;
38             release(&ftable.lock);
39             return f;
40         }
41     }
42     release(&ftable.lock);
43     return 0;
44 }
```

- **filealloc()** allocate a new open file structure from the global file table.
- **filealloc()** scans this table for a **struct file** with **ref == 0**.
 - It returns an unused entry, ready to be assigned to a file descriptor.
- The returned struct file will have its ref incremented upon use.

Manage file reference count

```
$ vim kernel/file.c
```

```
47 struct file*
48 filedup(struct file *f)
49 {
50     acquire(&ftable.lock);
51     if(f->ref < 1)
52         panic("filedup");
53     f->ref++;
54     release(&ftable.lock);
55     return f;
56 }
...
59 void
60 fileclose(struct file *f)
61 {
...
71     ff = *f;
72     f->ref = 0;
73     f->type = FD_NONE;
74     release(&ftable.lock);
75
76     if(ff.type == FD_PIPE){
77         pipeclose(ff.pipe, ff.writable);
78     } else if(ff.type == FD_INODE || ff.type == FD_DEVICE){
79         begin_op();
80         iput(ff.ip);
81         end_op();
82     }
83 }
```

- The **filedup()** function **increments the reference count** of **struct file**, allowing multiple file descriptors to refer to the same open file.
- The **fileclose()** function **decrements the reference count** of a struct file.
 - If the reference count reaches **zero**, it release the associated resource, whether it is an inode or a pipe.

File operation

- **fileread()** reads data from an open file.
 - If the file uses an inode, reads from the disk at the current offset.
 - If the file is a pipe, reads without using an offset.
- **filewrite()** writes data to an open file.
 - If the file uses an inode, writes data at the current offset.
 - If the file is a pipe, writes data directly, without using an offset.

System Calls for File-System

open, read, write system calls

open system call

```
$ vim kernel/sysfile.c
```

```
304  uint64
305  sys_open(void)
306  {
317      begin_op();
318
319      if(omode & O_CREATE){
320          ip = create(path, T_FILE, 0, 0);
321          if(ip == 0){
322              end_op();
323              return -1;
324          }
325      } else {
326          if((ip = namei(path)) == 0){
327              end_op();
328              return -1;
329          }
330          ilock(ip);
331          if(ip->type == T_DIR && omode != O_RDONLY){
332              iunlockput(ip);
333              end_op();
334              return -1;
335          }
336      }
```

- **sys_open()** function opens an existing file or creates a new one if necessary, and links it to the process's file descriptor table for subsequent access.
- After starting transaction, if **O_CREATE** is set, calls **create()** to allocate a new inode and initialize it.
 - Otherwise, uses **namei()** to look up the file by path.

open system call

```
$ vim kernel/sysfile.c

305 sys_open(void)
306 {
...
352     if(ip->type == T_DEVICE){
353         f->type = FD_DEVICE;
354         f->major = ip->major;
355     } else {
356         f->type = FD_INODE;
357         f->off = 0;
358     }
359     f->ip = ip;
360     f->readable = !(omode & O_WRONLY);
361     f->writable = (omode & O_WRONLY) || (omode & O_RDWR);
362
363     if((omode & O_TRUNC) && ip->type == T_FILE){
364         itrunc(ip);
365     }
366
367     iunlock(ip);
368     end_op();
369
370     return fd;
371 }
```

- Next, **initialize the file structure**.
 - Sets file type(device or inode), offset, inode pointer.
 - Sets read/write permissions according to **omode**.
- Then, truncate the file by calling **itrunc()**.
- Finally, ends the transaction and return the allocated file descriptor.

Create new inode

```
$ vim kernel/sysfile.c

245 static struct inode*
246 create(char *path, short type, short major, short minor)
247 {
248     struct inode *ip, *dp;
249     char name[DIRSIZ];
250
251     if((dp = nameiparent(path, name)) == 0)
252         return 0;
253
254     ilock(dp);
255
256     if((ip = dirlookup(dp, name, 0)) != 0){
257         iunlockput(dp);
258         ilock(ip);
259         if(type == T_FILE && (ip->type == T_FILE || ip->type == T_DEVICE))
260             return ip;
261         iunlockput(ip);
262         return 0;
263     }
264
265     if((ip = ialloc(dp->dev, type)) == 0){
266         iunlockput(dp);
267         return 0;
268     }
269
270     ...
302 }
```

- **create()** creates a new inode and directory entry for a file, directory, or device at the specified path.
- First, Locate parent directory .
- Next, check if entry already exists.
 - by using **dirlookup()** to check if name exists in the parent.
- Then allocate and initialize new node by using **ialloc()**.

Create new inode

```
$ vim kernel/sysfile.c

246 create(char *path, short type, short major, short minor)
...
270     ilock(ip);
271     ip->major = major;
272     ip->minor = minor;
273     ip->nlink = 1;
274     iupdate(ip);
275
276     if(type == T_DIR){
277         // No ip->nlink++ for ".": avoid cyclic ref count.
278         if(dirlink(ip, ".", ip->inum) < 0 || dirlink(ip, "..", dp->inum) < 0)
279             goto fail;
280     }
281
282     if(dirlink(dp, name, ip->inum) < 0)
283         goto fail;
284
285     if(type == T_DIR){
286         // now that success is guaranteed:
287         dp->nlink++;
288         iupdate(dp);
289     }
290
291     iunlockput(dp);
292
293     return ip;
...
302 }
```

- If directory, create . and .. entries by **dirlink()**.
- Finally, add entry to parent directory.
 - Adds the new entry(**name**) to the parent using **dirlink()**.
 - If it's a directory, also increments **dp->nlink** for the .. reference.

read, write system call

```
$ vim kernel/sysfile.c
```

```
68  uint64
69  sys_read(void)
70  {
...
75      argaddr(1, &p);
76      argint(2, &n);
77      if(argfd(0, 0, &f) < 0)
78          return -1;
79      return fileread(f, p, n);
80  }
81
82  uint64
83  sys_write(void)
84  {
...
89      argaddr(1, &p);
90      argint(2, &n);
91      if(argfd(0, 0, &f) < 0)
92          return -1;
93      return filewrite(f, p, n);
94  }
```

- `sys_read()`, `sys_write()` functions just call `fileread`, `filewrite`.

Putting the layers together

- `open("/d/f", O_CREATE)`
 - `sys_open` → `create` / `namei` (File descriptor → Pathname)
 - `namex` + `skipelem` walk the path (Pathname)
 - `dirlookup` / `dirlink` resolve names (Directory)
 - `ialloc` · `iget` · `ilock` obtain the inode (Inode)
 - `filealloc` + `fdalloc` return the fd (File descriptor)
- `read(fd, ...)` / `write(fd, ...)`
 - `fileread` / `filewrite` (File descriptor)
 - `readi` / `writeri` → `bmap` (Inode)
 - `bread` / `log_write` (Buffer cache / Logging)
 - `virtio_disk_rw` (Disk)

Thank you

